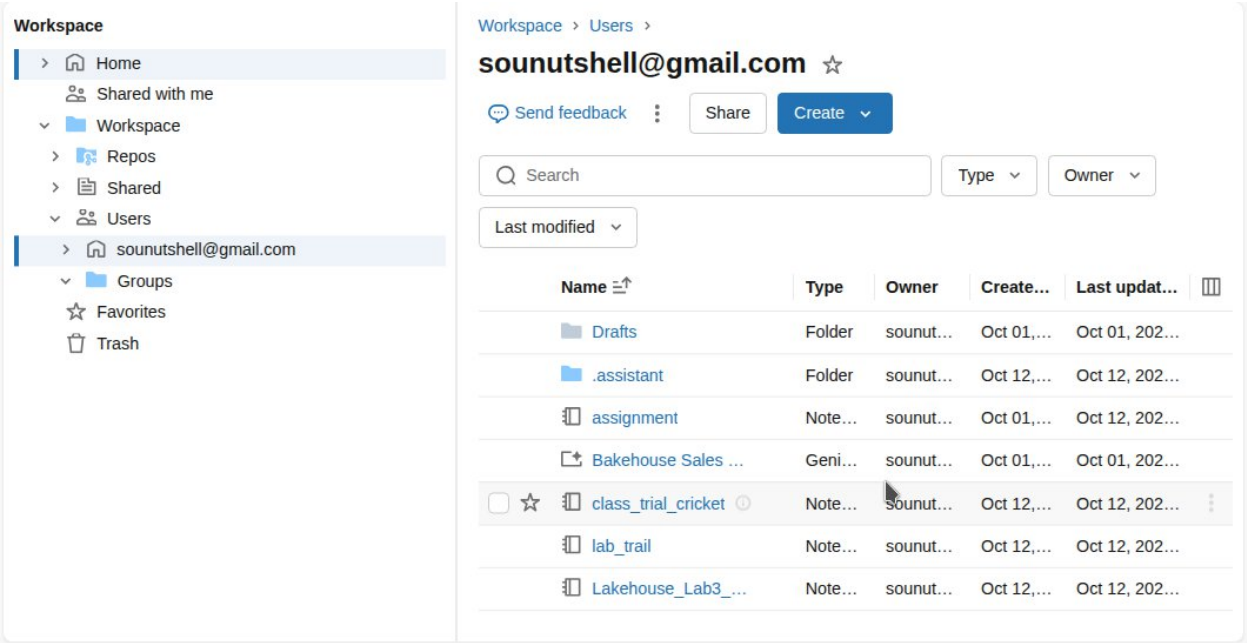
Soumiya_Ayyappan_W9_D1

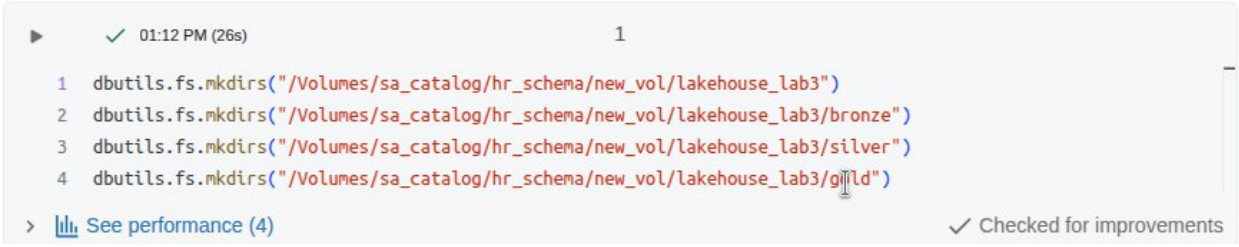
Part A:

Step 1:



Step 2:

Code:



Output:



Part B:

Step 3:

Code:

```
01:39 PM (5s)                                    2                    Python
1  import pandas as pd
2  url="https://raw.githubusercontent.com/databricks/Spark-The-Definitive-Guide/master/data/retail-data/all/
   online-retail-dataset.csv"
3  # bronze_path="/Volumes/sa_catalog/hr_schema/new_vol/lakehouse_lab3/bronze"
4  # dbutils.fs.cp(url, f"{bronze_path}/online-retail-dataset.csv")
5  df=pd.read_csv(url)
6  # df_raw=spark.read.option("inferSchema","True").option("header","True").csv(url)
7  #f"{bronze_path}/online-retail-dataset.csv")
8  df_raw=spark.createDataFrame(df)
9  display(df_raw)
10
```

Output:

See performance (1) Checked for improvements

2 dataframes: df, df_raw

Table

	InvoiceNo	StockCode	Description	Quantity	Invoice
1	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8
2	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8
3	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8
4	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8
5	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8
6	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	12/1/2010 8
7	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	12/1/2010 8
8	536366	22633	HAND WARMER UNION JACK	6	12/1/2010 8
9	536366	22632	HAND WARMER RED POLKA DOT	6	12/1/2010 8
10	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	12/1/2010 8
11	536367	22745	POPPY'S PLAYHOUSE BEDROOM	6	12/1/2010 8
12	536367	22748	POPPY'S PLAYHOUSE KITCHEN	6	12/1/2010 8

Step 4:

Code:

```
Just now (7s)                                    3                    Python
1  df_raw.write.format("delta").mode("overwrite").saveAsTable("bronze_retail")
2  display(spark.sql("select * from bronze_retail limit 10"))
> See performance (2)                                  Checked for improvements
```

Output:

Table

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate
1	576840	21993	FLORAL FOLK STATIONERY SET	1	11/16/2011 15:23
2	576840	22059	CERAMIC STRAWBERRY DESIGN MUG	1	11/16/2011 15:23
3	576840	22065	CHRISTMAS PUDDING TRINKET POT	4	11/16/2011 15:23
4	576840	22069	BROWN PIRATE TREASURE CHEST	1	11/16/2011 15:23
5	576840	22074	6 RIBBONS SHIMMERING PINKS	1	11/16/2011 15:23
6	576840	22075	6 RIBBONS ELEGANT CHRISTMAS	4	11/16/2011 15:23
7	576840	22076	6 RIBBONS EMPIRE	1	11/16/2011 15:23
8	576840	22083	PAPER CHAIN KIT RETROSPOT	1	11/16/2011 15:23
9	576840	22086	PAPER CHAIN KIT 50'S CHRISTMAS	4	11/16/2011 15:23
10	576840	22089	PAPER BUNTING VINTAGE PAISLEY	1	11/16/2011 15:23

Part C

Step 5:

Code:

02:25 PM (<1s) 3 Python

```
df_silver=df_raw.dropna()
df_silver=df_silver.withColumnRenamed("InvoiceNo","invoice_no").withColumnRenamed("Description","description")
from pyspark.sql.functions import col
df_silver=df_silver.filter(col("Quantity")>0
```

Output:

```
df_silver: pyspark.sql.connect.dataframe.DataFrame = [invoice_no: string, StockCode: string ... 6 more fields]
```

Step 6:

Code:

02:37 PM (8s) 5 Python

```
df_silver.write.format("delta").mode("overwrite").saveAsTable("silver_retail")
display(spark.sql("select count(*) from silver_retail"))
```

See performance (2) Checked for improvements

Output:

Table +

	count(*)
1	397924

Part D

Step 7:

Code:

```
02:46 PM (13s)   6   Python

1 from pyspark.sql.functions import sum,col,to_date
2 df_gold=df_silver.withColumn("date",to_date("InvoiceDate", "M/d/yyyy H:mm")).groupBy("date").agg(sum(col
  ("Quantity")*col("UnitPrice")).alias("daily_revenue"))
3 df_gold.write.format("delta").mode("overwrite").saveAsTable("gold_daily_revenue")
4 display(spark.sql("select * from gold_daily_revenue"))

> See performance (2)   Checked for improvements
```

Output:

Table +

	date	daily_revenue
1	2010-12-06	31215.64000000016
2	2010-12-22	4886.519999999998
3	2011-01-20	15244.73000000001
4	2011-01-21	24279.76
5	2011-01-30	6615.749999999991
6	2011-02-02	17145.230000000058
7	2011-01-23	10400.249999999967
8	2011-01-26	17490.820000000043
9	2011-01-24	17632.370000000024

Step 8:

Code:

```
02:49 PM (2s)

1  %sql
2  select * from gold_daily_revenue order by date;

See performance (1)
```

Output:

